

Создание элементов из массива на JavaScript

Пусть у нас есть некоторый массив:

```
1 | let arr = [1, 2, 3, 4, 5];
```

Пусть также у нас есть некоторый элемент-родитель:

```
1 | <div id="parent"></div>
```

Давайте добавим в наш родитель новые абзацы, текстом которых будут элементы нашего массива.

Реализуем описанное:

```
1 | let parent = document.querySelector('#parent');
2 |
3 | let arr = [1, 2, 3, 4, 5];
4 |
5 | for (let elem of arr) {
6 |   let p = document.createElement('p');
7 |   p.textContent = elem;
8 |
9 |   parent.appendChild(p);
10 | }
```

№1

©jsPmPrECVA

Модифицируйте мой код так, чтобы по клику на абзац к его содержимому прибавлялась единица.

Практика на создание списков ul на JavaScript

№1

©jsPmPrLC

Дан ul:

```
1 | <ul id="elem"></ul>
```

Дан массив. Вставьте элементы этого массива в конец ul так, чтобы каждый элемент стоял в своем li.

№2

©jsPmPrLC

Модифицируйте предыдущую задачу так, чтобы по клику на любой из вставленных элементов на экран выводился текст этого элемента.

№3

©jsPmPrLC

Модифицируйте предыдущую задачу так, чтобы по клику на li ей в конец добавлялся '!'.
1

Модифицируйте предыдущую задачу так, чтобы по повторное нажатие на li не приводило к добавлению второго '!'.

Создание HTML таблиц на JavaScript

Пусть у нас есть вот такая пустая HTML таблица:

```
1 <table id="table"></table>
```

Давайте заполним эту таблицу рядами и колонками. Вот пример того, что у нас должно получиться:

```
1 <table id="table">
2   <tr>
3     <td></td>
4     <td></td>
5     <td></td>
6   </tr>
7   <tr>
8     <td></td>
9     <td></td>
10    <td></td>
11  </tr>
12  <tr>
13    <td></td>
14    <td></td>
15    <td></td>
16  </tr>
17 </table>
```

Для решения задачи нам нужно два вложенных цикла. Первый цикл будет создавать ряды таблицы, а второй - ячейки в каждом ряду:

```
1 let table = document.querySelector('#table');
2
3 for (let i = 0; i < 3; i++) {
4   let tr = document.createElement('tr');
5
6   for (let i = 0; i < 3; i++) {
7     let td = document.createElement('td');
8     tr.appendChild(td);
9   }
10
11   table.appendChild(tr);
12 }
```

Дана пустая HTML таблица. С помощью двух вложенных циклов for заполните эту таблицу 5-ю рядами с 5-ю колонками.

№2

⊗jsPmPrTC

Модифицируйте предыдущую задачу так, чтобы таблица создавалась размером 10 рядов на 5 колонок.

№3

⊗jsPmPrTC

Модифицируйте предыдущую задачу так, чтобы в каждую td добавлялся текст 'x'.

№4

⊗jsPmPrTC

Реализуйте генератор таблиц, ширина и высота таблиц задается в двух инпутах (например, таблица 5 на 10 ячеек).

Последовательное заполнение HTML таблиц

Пусть у нас есть вот такая пустая HTML таблица:

```
1 <table id="table"></table>
```

Давайте заполним эту таблицу ячейками и сделаем так, чтобы в этих ячейках шли числа от 1 до 9. Вот пример того, что у нас должно получиться:

```
1 <table id="table">
2   <tr>
3     <td>1</td>
4     <td>2</td>
5     <td>3</td>
6   </tr>
7   <tr>
8     <td>4</td>
9     <td>5</td>
10    <td>6</td>
11  </tr>
12  <tr>
13    <td>7</td>
14    <td>8</td>
15    <td>9</td>
16  </tr>
17 </table>
```

Давайте приступим к реализации.

Для начала давайте просто сделаем таблицу размером 3 на 3, заполненную буквами 'x':

```
1 let table = document.querySelector('#table');
2
3 for (let i = 0; i < 3; i++) {
4   let tr = document.createElement('tr');
5
6   for (let i = 0; i < 3; i++) {
7     let td = document.createElement('td');
8     td.textContent = 'x';
9     tr.appendChild(td);
10  }
11
12  table.appendChild(tr);
13 }
```

Давайте теперь сделаем так, чтобы вместо букв 'x' в ячейки записывались числа по возрастанию.

Для этого нам необходимо ввести еще один счетчик, который будет последовательно увеличивать свои значения в каждой итерации внутреннего цикла, вот так:

```
1 let table = document.querySelector('#table');
2
3 let k = 1; // начальное значение счетчика
4 for (let i = 0; i < 3; i++) {
5   let tr = document.createElement('tr');
6
7   for (let i = 0; i < 3; i++) {
8     let td = document.createElement('td');
9
10    td.textContent = k; // записываем счетчик в ячейку
11    k++; // увеличиваем счетчик
12
13    tr.appendChild(td);
14  }
15
16  table.appendChild(tr);
17 }
```

№1

©jsPmPrTCF

Выведите на экран HTML таблицу размером 5 рядов на 5 колонок так, чтобы в ее ячейках последова-

№2

©jsPmPrTCF

Модифицируйте предыдущую задачу так, чтобы в ячейках таблицы были записаны четные числа в промежутке от 2 до 50.

Создание HTML таблицы из массива на JavaScript

Пусть у нас есть некоторая пустая таблица:

```
1 <table id="table"></table>
```

Пусть у нас также дан некоторый двумерный массив, например, такой:

```
1 let arr = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]];
```

Давайте на основе этого массива сделаем HTML таблицу, заполненную элементами данного массива. То есть у нас получится таблица с тремя рядами, в каждом из которых будет по 4 ячейки.

Решим задачу с помощью двух вложенных циклов for-of:

```
1 let arr = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]];
2 let table = document.querySelector('#table');
3
4 for (let subArr of arr) {
5   let tr = document.createElement('tr');
6
7   for (let elem of subArr) {
8     let td = document.createElement('td');
9     td.textContent = elem;
10    tr.appendChild(td);
11  }
12
13  table.appendChild(tr);
14 }
```

Как вы видите, наше решение получилось универсальным и не зависит от количества подмассивов и количества элементов в каждом подмассиве. Единственное условие - чтобы в подмассивах было одинаковое количество элементов.

№1

⊗jsPmPrTCVA

Дан массив:

```
1 let arr = [[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]];
```

Не подсматривая в теоретическую часть урока, выведите элементы приведенного массива в виде HTML таблицы table.

№2

⊗jsPmPrTCVA

Модифицируйте предыдущую задачу так, чтобы в таблицу записывались не элементы, а квадраты этих элементов.

Создание HTML таблицы из массива объектов

Пусть у нас дан некоторый массив объектов, например, вот такой массив с юзерами:

```
1 let users = [  
2   {  
3     name: 'name1',  
4     surname: 'surname1',  
5     patronymic: 'patronymic1'  
6   },  
7   {  
8     name: 'name2',  
9     surname: 'surname2',  
10    patronymic: 'patronymic2'  
11  },  
12  {  
13    name: 'name3',  
14    surname: 'surname3',  
15    patronymic: 'patronymic3'  
16  },  
17 ];
```

Давайте сделаем из этого массива HTML таблицу, в каждый ряд которой запишем данные отдельного юзера.

При такой структуре хранения данных обычно запускают один цикл по массиву и создают каждую ячейку таблицы вручную, вот так:

```
1 <table id="table"></table>  
  
1 let table = document.getElementById('table');  
2  
3 for (let user of users) {  
4   let tr = document.createElement('tr');  
5  
6   let td1 = document.createElement('td');  
7   td1.textContent = user.name;  
8   tr.appendChild(td1);  
9  
10  let td2 = document.createElement('td');  
11  td2.textContent = user.surname;  
12  tr.appendChild(td2);  
13  
14  let td3 = document.createElement('td');  
15  td3.textContent = user.patronymic;  
16  tr.appendChild(td3);  
17  
18  table.appendChild(tr);  
19 }
```

Такой подход дает большую гибкость - мы можем сами регулировать порядок расположения данных по ячейкам таблицы (можем, к примеру, поменять местами имя и фамилию).

Кроме того, при желании мы можем навешивать события на определенные ячейки. К примеру, можно на ячейку с фамилией навесить какое-то действие по клику и так далее.

№ 1

©jsPmPrTCVOA

Дан следующий массив с работниками:

```
1 let employees = [  
2   {name: 'employee1', age: 30, salary: 400},  
3   {name: 'employee2', age: 31, salary: 500},  
4   {name: 'employee3', age: 32, salary: 600},  
5 ];
```

Выведите элементы этого массива в виде HTML таблицы.

№ 2

©jsPmPrTCVOA

Модифицируйте предыдущую задачу так, чтобы по клику на любую ячейку с возрастом ее содержимое увеличивалось на 1.

№ 3

©jsPmPrTCVOA

Модифицируйте предыдущую задачу так, чтобы по клику на любую ячейку с зарплатой ее содержимое увеличивалось на 10%.

Добавление рядов и колонок в HTML таблицу

Пусть у нас есть некоторая HTML таблица `#table`. Давайте научимся добавлять в нее новые ряды и колонки.

Добавление рядов

Добавление рядов не составляет труда: нужно создать `tr`, а затем запустить цикл, который добавит нужное количество ячеек в этот ряд (пусть 3):

```
1 let table = document.querySelector('#table');  
2  
3 let tr = document.createElement('tr');  
4  
5 for (let i = 1; i <= 3; i++) {  
6   let td = document.createElement('td');  
7   tr.appendChild(td);  
8 }  
9  
10 table.appendChild(tr);
```

№ 1

©jsPmPrTRCA

Сделайте кнопку, по нажатию на которую в таблицу будет добавляться новый ряд.

Добавление колонок

А вот с добавлением колонок чуть посложнее: нужно запустить цикл, который переберет все ряды таблицы и в каждый ряд добавит новую ячейку:

```
1 let trs = document.querySelectorAll('#table tr');
2
3 for (let tr of trs) {
4   let td = document.createElement('td');
5   tr.appendChild(td);
6 }
```

№2

©jsPmPrTRCA

Дана таблица размером 2 на 2:

```
1 <table id="table">
2   <tr>
3     <td></td>
4     <td></td>
5   </tr>
6   <tr>
7     <td></td>
8     <td></td>
9   </tr>
10 </table>
```

```
1 td {
2   width: 50px;
3   height: 50px;
4   border: 1px solid black;
5 }
```

Дана также кнопка. Сделайте так, чтобы по нажатию на кнопку таблица увеличивалась на один ряд и на одну колонку.

Изменение ячеек HTML таблицы на JavaScript

Пусть дана некоторая HTML таблица #table. Пусть перед нами стоит задача что-то сделать с каждой ячейкой таблицы, например, записать в каждую из них текст '!'.
Давайте обсудим нюансы решения подобной задачи.

Как вы знаете, HTML таблица имеет двухмерную структуру: есть ряды, а в них ячейки. Можно решить нашу задачу следующим образом: перебрать циклом ряды и в каждом ряду перебрать циклом ячейки и сделать с ними нужное нам действие. То есть приведенная схема решения подобна тому, как бы мы создавали такую таблицу, заполняя ее рядами и ячейками.

Однако, в данном случае, два вложенных цикла будут лишними: можно просто получить все td и перебрать их циклом, выполняя нужную операцию. Сделаем это:

```
1 let tds = document.querySelectorAll('#table td');
2
3 for (let td of tds) {
4   td.textContent = '!';
5 }
```

№ 1

©jsPmPrTCM

Пусть дана некоторая HTML таблица с числами и кнопка. По нажатию на кнопку увеличьте число в каждой ячейки таблицы в два раза.

Самоудаление новых элементов на JavaScript

В предыдущем уроке мы в вами научились делать так, чтобы элементы удаляли сами себя по клику.

Пусть теперь элементов в родителе изначально нет:

```
1 <div id="parent"></div>
```

Давайте в цикле создадим 9 новых абзацев, при этом сделаем так, чтобы любой абзац удалялся по клику на нем:

```
1 let parent = document.querySelector('#parent');
2
3 for (let i = 1; i <= 9; i++) {
4   let p = document.createElement('p');
5   p.textContent = i;
6
7   p.addEventListener('click', function() {
8     this.remove();
9   });
10
11   parent.appendChild(p);
12 }
```

№ 1

©jsPmPrNER

Дан следующий код:

```

1 <ul id="parent">
2   <li>1</li>
3   <li>2</li>
4   <li>3</li>
5 </ul>
6
7 <input type="submit" id="button">

```

Сделайте так, чтобы по клику на кнопку в список добавлялся новый элемент. Сделайте так, чтобы любая li удалялась по клику на нее. Речь идет как о тех li, которые уже есть в списке, так о новых, созданных после нажатия на кнопку.

Ссылка на удаление элемента на JavaScript

Пусть у нас абзац. Давайте рядом с ним сделаем ссылку, с помощью которой этот абзац можно будет удалить.

Реализуем:

```

1 <div id="parent">
2   <p id="elem">text</p>
3   <a href="#" id="remove">remove</a>
4 </div>

1 let elem = document.querySelector('#elem');
2 let remove = document.querySelector('#remove');
3
4 remove.addEventListener('click', function() {
5   elem.remove();
6 });

```

Обратите внимание на то, что в атрибуте href ссылки стоит #. Если эту решетку убрать - мы получим переход по ссылке и, как следствие, обновление страницы.

На самом деле удаление абзаца тоже будет происходить, но мы это не заметим, так как страница обновится и все вернется в изначальное положение.

Для решения проблемы нужно предотвратить переход по ссылке с помощью preventDefault:

```

1 <div id="parent">
2   <p id="elem">text</p>
3   <a href="" id="remove">remove</a>
4 </div>

1 let elem = document.querySelector('#elem');
2 let remove = document.querySelector('#remove');
3
4 remove.addEventListener('click', function(event) {
5   elem.remove();
6   event.preventDefault(); // отменяем переход по ссылке
7 });

```

№ 1

⊗jsPmPrRL

Самостоятельно, не подсматривая в мой код, решите описанную задачу.

Создание ссылок для удаление элементов на JavaScript

Пусть теперь у нас есть много абзацев:

```
1 <div id="parent">
2   <p>text1</p>
3   <p>text2</p>
4   <p>text3</p>
5 </div>
```

Давайте сделаем так, чтобы каждому абзацу в конец добавлялась ссылка на его удаление.

Для начала давайте просто реализуем добавление ссылок:

```
1 let elems = document.querySelectorAll('#parent p');
2
3 for (let elem of elems) {
4   let remove = document.createElement('a');
5   remove.href = '';
6   remove.textContent = 'remove';
7
8   elem.appendChild(remove);
9 }
```

Давайте теперь сделаем так, чтобы по нажатию на ссылку, удалялся соответствующий ей абзац:

```
1 let elems = document.querySelectorAll('#parent p');
2
3 for (let elem of elems) {
4   let remove = document.createElement('a');
5   remove.href = '';
6   remove.textContent = 'remove';
7   elem.appendChild(remove);
8
9   remove.addEventListener('click', function(event) {
10     elem.remove();
11     event.preventDefault();
12   });
13 }
```

№ 1

⊗jsPmPrCRL

Дан тег ul. Добавьте в конец каждого тега li ссылку на удаления этого li из списка.

№ 2

⊗jsPmPrCRL

Дана HTML таблица. Добавьте в нее еще одну колонку, в которой для каждого ряда таблицы будет стоять ссылка на удаление этого ряда.

Редактирование отдельного элемента на JavaScript

Сейчас мы научимся редактировать тексты элементов с помощью полей ввода. Начнем с простых вещей и будем постепенно усложнять.

Итак, пусть у нас даны абзац и инпут, расположенные в одном родителе:

```
1 <div id="parent">
2   <p id="elem">text</p>
3   <input id="input">
4 </div>
```

Давайте сделаем так, чтобы по потере фокуса в инпуте его текст появлялся в абзаце:

```
1 let elem = document.querySelector('#elem');
2 let input = document.querySelector('#input');
3
4 input.addEventListener('blur', function() {
5   elem.textContent = this.value;
6 });
```

Давайте теперь сделаем так, чтобы по заходу на страницу в инпуте уже стоял текст абзаца. Таким образом мы с помощью инпута сможем редактировать текст, расположенный в абзаце.

Реализуем:

```
1 let elem = document.querySelector('#elem');
2 let input = document.querySelector('#input');
3
4 input.value = elem.textContent; // записываем в инпут текст абзаца
5
6 input.addEventListener('blur', function() {
7   elem.textContent = this.value;
8 });
```

№1

©jsPmPrSEE

Модифицируйте приведенный выше код так, чтобы текст абзаца менялся не по потере фокуса, а по мере ввода текста в инпут.

Появление инпута

Давайте теперь сделаем так, чтобы инпута изначально не было на странице, а он появлялся по клику на абзац. То есть наш начальный HTML будет выглядеть так:

```
1 <div id="parent">
2   <p id="elem">text</p>
3 </div>
```

Для начала просто реализуем появление инпута, без редактирования:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function() {
4   let input = document.createElement('input');
5   input.value = elem.textContent;
6
7   elem.parentElement.appendChild(input);
8 });
```

А теперь давайте сделаем так, чтобы по потере фокуса в инпуте менялся текст абзаца:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function() {
4   let input = document.createElement('input');
5   input.value = elem.textContent;
6
7   input.addEventListener('blur', function() {
8     elem.textContent = this.value;
9   });
10
11   elem.parentElement.appendChild(input);
12 });
```

Наш код, однако, несовершенен, так как каждое нажатие на абзац будет приводить к появлению нового инпута.

Для решения проблемы просто будем по потере фокуса удалять текущий инпут:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function() {
4   let input = document.createElement('input');
5   input.value = elem.textContent;
6
7   input.addEventListener('blur', function() {
8     elem.textContent = this.value;
9     this.remove(); // удалим инпут
10  });
11
12   elem.parentElement.appendChild(input);
13 });
```

Прячем текст при редактировании элемента на JavaScript

Давайте теперь сделаем так, чтобы при редактировании инпут появлялся в самом абзаце - вместо текста этого абзаца. Пусть затем при окончании редактирования инпут будет убираться, а вместо него опять появляться текст абзаца.

Приступим к реализации.

Для начала давайте сделаем так, чтобы по клику на абзац ему в конец добавлялся инпут с текстом этого абзаца:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function() {
4   let input = document.createElement('input');
5   input.value = elem.textContent;
6
7   elem.appendChild(input);
8 });
```

Наш код, однако, очень несовершенен - каждый раз по клику на абзац в него будет добавляться новый инпут.

При этом, если в первом инпуте будет стоять текст абзаца, то во втором инпуте уже будет стоять текст абзаца вместе с первым инпутом, а третьем инпуте уже будет стоять текст абзаца вместе с двумя инпутами и так далее.

Обратите также внимание на то, что клик по добавленному инпуту будет расцениваться как клик по абзацу: дело в том, что инпут находится в абзаце, и клик по инпуту просто всплывет выше к этому абзацу.

Это приведет к тому, что после появления первого инпута при попытке нажать на него для того, чтобы начать редактирование, мы автоматически сделаем клик по абзацу со всеми вытекающими последствиями.

Итак, проблема обрисована. Давайте теперь ее исправим.

Для этого просто при появлении инпута отвяжем от абзаца обработчик клика. В этом случае только первый клик по абзацу будет приводить к появлению инпута, а остальные клики, сделанные после появления инпута, будут проигнорированы.

Реализуем:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function func() {
4   let input = document.createElement('input');
5   input.value = elem.textContent;
6
7   elem.appendChild(input);
8
9   elem.removeEventListener('click', func); // отвяжем событие
10 });
```

Давайте теперь сделаем так, чтобы при появлении инпута, текст самого абзаца исчезал. Для этого перед вставкой инпута `textContent` абзаца присвоим пустой строке:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function func() {
4   let input = document.createElement('input');
5
6   input.value = elem.textContent; // сначала записываем текст абзаца в инпут
7   elem.textContent = ''; // затем убираем текст абзаца
8   elem.appendChild(input); // затем вставляем инпут
9
10  elem.removeEventListener('click', func);
11 });
```

Давайте теперь сделаем так, чтобы при потере фокуса в инпуте текст этого инпута записывался в абзац:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function func() {
4   let input = document.createElement('input');
5
6   input.value = elem.textContent;
7   elem.textContent = '';
8   elem.appendChild(input);
9
10  input.addEventListener('blur', function() {
11    elem.textContent = this.value;
12  });
13
14  elem.removeEventListener('click', func);
15 });
```

Обратите внимание на то, что нам не нужно удалять инпут - он удаляет сам себя, когда записывает свой текст в абзац: так как инпут является частью текста абзаца, то запись какого-то текста в этот абзац просто удаляет наш инпут, и все.

У нас, однако, есть еще одна проблема: текст абзаца будет редактироваться лишь первый раз. После первого редактирования повторное нажатие на текст абзаца ни к чему не приведет.

Дело в том, что в момент появления инпута мы отвязали событие от абзаца по описанным выше причинам. Теперь нам необходимо в момент окончания редактирования привязать событие обратно.

Сделаем это:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function func() {
4   let input = document.createElement('input');
5   input.value = elem.textContent;
6
7   elem.textContent = '';
8   elem.appendChild(input);
9
10  input.addEventListener('blur', function() {
11    elem.textContent = this.value;
12    elem.addEventListener('click', func); // повесим событие обратно
13  });
14
15  elem.removeEventListener('click', func);
16 });
```

№1

©jsPmPrEHт

Самостоятельно, не подсматривая в мой код, решите описанную задачу.

Редактирование в группе элементов на JavaScript

Пусть теперь у нас есть не один абзац, а много:

```
1 <p>text1</p>
2 <p>text2</p>
3 <p>text3</p>
```

Давайте сделаем так, чтобы по клику на любой абзац в нем появлялся инпут для редактирования.

На самом деле такая задача для нас не представляет сложности, так как почти весь код был получен нами в предыдущем уроке.

Для решения нашей задачи просто запустим цикл по абзацам и в цикле используем код предыдущего урока (этот код даже не понадобится изменять):

```
1 let elems = document.querySelectorAll('p');
2
3 for (let elem of elems) {
4   elem.addEventListener('click', function func() {
5     let input = document.createElement('input');
6     input.value = elem.textContent;
7
8     elem.textContent = '';
9     elem.appendChild(input);
10
11    input.addEventListener('blur', function() {
12      elem.textContent = this.value;
13      elem.addEventListener('click', func);
14    });
15
16    elem.removeEventListener('click', func);
17  });
18 }
```

№1

⊗jsPmPrEGE

Дан тег `ul`. Сделайте так, чтобы по клику на любую `li` в ней появлялся инпут, с помощью которого можно будет отредактировать текст этой `li`.

№2

⊗jsPmPrEGE

Дана HTML таблица. Сделайте так, чтобы по клику на любую ячейку в ней появлялся инпут для редактирования текста этой ячейки.

Одновременное редактирование и удаление элементов

Пусть у нас есть некоторый набор абзацев:

```
1 <div id="parent">
2   <p>text1</p>
3   <p>text2</p>
4   <p>text3</p>
5 </div>
```

Давайте для этих абзацев сделаем так, чтобы можно было редактировать текст каждого абзаца и при этом в конце каждого абзаца стояла ссылка на удаление.

При реализации описанной задачи нас ждет некоторая проблема.

Для того, чтобы понять суть проблемы, давайте посмотрим на HTML код, который получится, когда в конец каждого абзаца будут добавлены ссылки на удаление:

```
1 <div id="parent">
2   <p>text1<a href="">remove</a></p>
3   <p>text2<a href="">remove</a></p>
4   <p>text3<a href="">remove</a></p>
5 </div>
```

Представим теперь, что по клику на любой абзац в нем будет появляться инпут для редактирования текста. В этом случае в инпут будет попадать весь текст абзаца - вместе со ссылкой на удаление!

Это, конечно же, не правильно.

Более удачным решение будет обернуть тексты абзацев в теги `span`, вот так:

```
1 <div id="parent">
2   <p><span>text1</span><a href="">remove</a></p>
3   <p><span>text2</span><a href="">remove</a></p>
4   <p><span>text3</span><a href="">remove</a></p>
5 </div>
```

Для такого кода можно просто навесить событие для редактирования не на сам абзац, а на `span` с текстом. В этом случае инпут для редактирования будет появляться в теге `span`, и наша ссылка для удаления останется нетронутой.

№ 1

⊗jsPmPrRET

Дан следующий HTML код:

```
1 <div id="parent">
2   <p><span>text1</span></p>
3   <p><span>text2</span></p>
4   <p><span>text3</span></p>
5 </div>
```

Добавьте ссылку на удаление в конец каждого абзаца.

Сделайте так, чтобы по клику на `span` в нем появлялся инпут для редактирования.

Пусть теперь изначально тегов `span` нет:

```
1 <div id="parent">
2   <p>text1</p>
3   <p>text2</p>
4   <p>text3</p>
5 </div>
```

Оберните сначала текст абзаца в теги `span`, добавьте к этим тегам возможность редактирования, а затем добавьте в конец каждого абзаца ссылку на удаление.

Стилизация элементов на JavaScript

Пусть у нас есть несколько абзацев:

```
1 <p>text1</p>
2 <p>text2</p>
3 <p>text3</p>
```

Давайте переберем эти абзацы циклом и в конец каждого абзаца добавим ссылку, по нажатию на которую тексту абзаца будет добавляться некоторая стилизация. К примеру, текст абзаца будет становиться перечеркнутым (это делает CSS свойство `text-decoration`).

Давайте обсудим два нюанса такой задачи.

Во-первых, как уже обсуждалось в предыдущих уроках, не следует менять стили абзацы напрямую через JavaScript - гораздо удобнее будет навешивать какие-нибудь CSS классы.

Во-вторых, при реализации такой задачи вас ожидает некоторый сюрприз. Чтобы понять его суть, давайте посмотрим на HTML код абзацев после добавления ссылок:

```
1 <p>text1<a href="">link</a></p>
2 <p>text2<a href="">link</a></p>
3 <p>text3<a href="">link</a></p>
```

Представим теперь, что по нажатию на ссылку текст абзаца перечеркнется. Однако, ссылка в данном случае также является частью абзаца и тоже перечеркнется! Скорее всего такой эффект нам не нужен. Мы хотим, чтобы текст перечеркивался, но ссылка - нет.

Для решения проблемы нужно просто обернуть текст абзаца в тег `span`, вот так:

```
1 <p><span>text1</span><a href="">link</a></p>
2 <p><span>text2</span><a href="">link</a></p>
3 <p><span>text3</span><a href="">link</a></p>
```

Дан следующий HTML код:

```
1 <p>text1</p>
2 <p>text2</p>
3 <p>text3</p>
```

Добавьте в конец каждого абзаца ссылку, по клику на которую текст абзаца будет перечеркиваться (а ссылка - нет).

№2

⊗jsPmPrES

Модифицируйте предыдущую задачу так, чтобы после нажатия на ссылку эта ссылка удалялась из абзаца (а текст абзаца становился перечеркнутым).

№3

⊗jsPmPrES

Дана некоторая HTML таблица. Добавьте в эту таблицу еще одну колонку со ссылкой. По нажатию на эту ссылку ряд с этой ссылкой должен стать зеленого фона.

№4

⊗jsPmPrES

Модифицируйте предыдущую задачу так, чтобы первое нажатие по ссылке красило ряд в зеленый фон, а второе нажатие отменяло это действие.

Кнопки для скрытия и показа элемента на JavaScript

В данном разделе мы научимся скрывать и показывать элементы страницы. Как обычно, начнем с чего-то простого, и будем постепенно усложнять.

Пусть для разминки у нас есть абзац и две кнопки:

```
1 <p id="elem">text</p>
2 <input type="submit" id="show" value="show">
3 <input type="submit" id="hide" value="hide">
```

Получим ссылки на наши элементы в переменные:

```
1 let elem = document.querySelector('#elem');
2 let show = document.querySelector('#show');
3 let hide = document.querySelector('#hide');
```

Давайте теперь сделаем так, чтобы по клику на одну кнопку наш абзац скрывался, а на другую - показывался. Будем для этого давать или убирать элементу соответствующий CSS класс:

```
1 .hidden {
2   display: none;
3 }
```

Решим нашу задачу:

```
1 hide.addEventListener('click', function() {
2   elem.classList.add('hidden');
3 });
4
5 show.addEventListener('click', function() {
6   elem.classList.remove('hidden');
7 });
```

№ 1

©jsPmPrHShB

Модифицируйте мой код так, чтобы была только одна кнопка. Пусть по первому клику на эту кнопку элемент показывается, а по второму - скрывается.

Много элементов с кнопками показа на JavaScript

Пусть теперь у нас есть много абзацев и у каждого своя кнопка для сокрытия:

```
1 <p>1</p><button>toggle</button>
2 <p>2</p><button>toggle</button>
3 <p>3</p><button>toggle</button>
```

Сделаем так, чтобы по клику на кнопку скрывался или показывался соответствующий ей абзац.

Для этого нам каким-то образом нужно связать кнопки с нашими абзацами. Для этого существует несколько способов.

Первый способ

Свяжем кнопки и абзацы следующим образом:

```
1 <p id="elem1">1</p><button data-elem="elem1">toggle</button>
2 <p id="elem2">2</p><button data-elem="elem2">toggle</button>
3 <p id="elem3">3</p><button data-elem="elem3">toggle</button>
```

Теперь по клику на любую кнопку мы будем читать содержимое ее атрибута data-elem и искать абзац с таким id. Его и будем тоглить. Реализуем описанное:

```
1 let buttons = document.querySelectorAll('button');
2
3 for (let button of buttons) {
4   button.addEventListener('click', function() {
5     let elem = document.querySelector('#' + this.dataset.elem);
6     elem.classList.toggle('hidden');
7   });
8 }
```

№ 1

©jsPmPrMEWShB

Изучите мое решение, а затем самостоятельно, не подсматривая в мой код, решите эту задачу.

Второй способ

Расставлять id и data-атрибуты не очень удобно. Давайте сделаем так, чтобы связь была по порядковому номеру: пусть первая кнопка скрывает первый абзац, вторая кнопка - второй и так далее.

Реализуем описанное:

```
1 let buttons = document.querySelectorAll('button');
2 let elems   = document.querySelectorAll('p');
3
4 for (let i = 0; i < buttons.length; i++) {
5   buttons[i].addEventListener('click', function() {
6     elems[i].classList.toggle('hidden');
7   });
8 }
```

№2

⊗jsPmPrMEWShB

Изучите мое решение, а затем самостоятельно, не подсматривая в мой код, решите эту задачу.

Третий способ

Как можно увидеть, абзац, связанный с кнопкой, является ее соседом слева. Можно это использовать в качестве связи:

```
1 let buttons = document.querySelectorAll('button');
2
3 for (let button of buttons) {
4   button.addEventListener('click', function() {
5     this.previousElementSibling.classList.toggle('hidden');
6   });
7 }
```

№3

⊗jsPmPrMEWShB

Изучите мое решение, а затем самостоятельно, не подсматривая в мой код, решите эту задачу.

Активация элементов на JavaScript

Пусть у нас есть некоторая HTML таблица #table с пустыми ячейками.

Давайте сделаем так, чтобы по нажатию на любую ячейку она каким-то образом *активировалась*, например, становилась красного фона. Для этого активированным ячейкам будем давать какой-нибудь CSS класс:

```
1 .active {
2   background: red;
3 }
```

Реализуем активацию:

```
1 let tds = document.querySelectorAll('#table td');
```

```

2
3 for (let td of tds) {
4   td.addEventListener('click', function() {
5     this.classList.add('active');
6   });
7 }

```

№1

⊗jsPmPrEA

Дана HTML список ul. Сделайте так, чтобы по нажатию на любой пункт списка он активировался красным фоном.

№2

⊗jsPmPrEA

Модифицируйте предыдущую задачу так, чтобы по нажатию на активированный пункт списка активация с него снималась.

Чередование стилей активации на JavaScript

В предыдущем уроке мы научились активировать ячейки по клику на них. Давайте теперь сделаем так, чтобы происходило чередование цветов: пусть первая ячейка красится в красный, вторая - в зеленый, третья - опять в красный и так далее.

Для решения этой задачи нам понадобится уже два CSS класса:

```

1 .color1 {
2   background: red;
3 }
4 .color2 {
5   background: green;
6 }

```

Реализуем описанное чередование:

```

1 let tds = document.querySelectorAll('#table td');
2
3 let color = 'color1';
4 for (let td of tds) {
5   td.addEventListener('click', function() {
6     if (color == 'color1') {
7       color = 'color2'
8     } else {
9       color = 'color1'
10    }

```

```
11 |  
12 |     this.classList.add(color);  
13 | });  
14 | }
```

№1

⊗jsPmPrASA

Разберите мой код, а затем самостоятельно повторите решение этой задачи.

Чередование многих цветов из массива

Пусть теперь мы хотим чередовать не два цвета, а произвольное количество. Давайте для этого будем хранить массив CSS классов:

```
1 | let colors = ['color1', 'color2', 'color3'];
```

Напишем цвета наших классов:

```
1 | .color1 {  
2 |     background: red;  
3 | }  
4 | .color2 {  
5 |     background: green;  
6 | }  
7 | .color3 {  
8 |     background: yellow;  
9 | }
```

А теперь реализуем чередование:

```
1 | let tds = document.querySelectorAll('#table td');  
2 |  
3 | let i = 0;  
4 | let colors = ['color1', 'color2', 'color3'];  
5 |  
6 | for (let td of tds) {  
7 |     td.addEventListener('click', function() {  
8 |         this.classList.add(colors[i]);  
9 |  
10 |         i++;  
11 |         if (i == colors.length) {  
12 |             i = 0;  
13 |         }  
14 |     });  
15 | }
```

№2

⊗jsPmPrASA

Расскажите, как работает приведенный мною код.

Практика на изменение элементов на JavaScript

№1

ⓧjsPmPrEChT

Дан массив. Выведите его элементы в виде списка `ul`.

№2

ⓧjsPmPrEChT

Модифицируйте предыдущую задачу так, чтобы по клику на любую `li` в ней появлялся инпут, с помощью которого ее можно будет отредактировать.

№3

ⓧjsPmPrEChT

Модифицируйте предыдущую задачу так, чтобы под списком был инпут, с помощью которого можно будет добавить новый элемент в конец списка `ul`. Сделайте так, чтобы новые `li` также можно было редактировать.

№4

ⓧjsPmPrEChT

Модифицируйте предыдущую задачу так, чтобы в конце каждой `li` стояла ссылка 'удалить', с помощью которой можно будет удалить эту `li` из `ul`.

№5

ⓧjsPmPrEChT

Модифицируйте предыдущую задачу так, чтобы в конце каждой `li` также стояла ссылка 'перечеркнуть', с помощью которой можно будет перечеркнуть текст данного тега `li`.

Массив объектов и таблица

№6

ⓧjsPmPrEChT

Дан следующий массив с работниками:

```
1 let employees = [  
2   {name: 'employee1', age: 30, salary: 400},  
3   {name: 'employee2', age: 31, salary: 500},  
4   {name: 'employee3', age: 32, salary: 600},  
5 ];
```


Выведите этих работников в HTML таблице.

№7

⊗jsPmPrEChT

Добавьте ячейкам созданной таблицы возможность редактирования.

№8

⊗jsPmPrEChT

Добавьте в вашу таблицу новую колонку со ссылкой на удаления ряда из таблицы.

№9

⊗jsPmPrEChT

Сделайте под таблицей 3 инпута и кнопку для добавление нового работника. Пусть в инпуты вводятся имя, возраст и зарплата, и по нажатию на кнопку новый работник добавляется в таблицу. Реализуйте редактирование ячеек для вновь добавленных работников.

Массив объектов и список

№10

⊗jsPmPrEChT

Дан следующий массив с работниками:

```
1 let employees = [  
2   {name: 'employee1', age: 30, salary: 400},  
3   {name: 'employee2', age: 31, salary: 500},  
4   {name: 'employee3', age: 32, salary: 600},  
5 ];
```

Выведите на экран каждого работника в своем теге li тег ul.

№11

⊗jsPmPrEChT

Сделайте так, чтобы по клику на имя, возраст или зарплату работника появлялся инпут для редактирования этого поля.

№ 12

©jsPmPrEChT

Добавьте в конец каждого тега `li` ссылку на удаление этого `li` из списка.

№ 13

©jsPmPrEChT

Под списком сделайте форму для добавления нового работника.